UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECJ2013

DATA STRUCTURES AND ALGORITHMS

SECTION 02, 2024/2025

ASSIGNMENT 2 REPORT

LECTURER'S NAME: DR ZURAINI ALI SHAH

GROUP 4

BIL	GROUP MEMBERS	MATRIC NO .
1	LUQMAN HARITH BIN MOHD SHAHAR	A23CS3009
2	LUVINESH SUDESH	A21EC0198
3	GOE JIE YING	A23CS0224
4	BASEL A. A. ABUNADA	A22EC4006

Table of Contents

Table of Contents **2**
Introduction **3**
Description of the stack operations. **3**
Chosen real-world application **3**
Comparative analysis of the implementations. **4**
Screenshots **5**
Array based approach screenshots 5
Pointer/Link list based approach screenshots 8

Introduction

The Last-In-First-Out (LIFO) principle means the last element added to the stack is the first one to be removed, opposite of the First-In-First-Out (FIFO) principle used in queues. A non-computing example of LIFO is stacking books vertically, where the last book placed on top is the first to be removed. In computing, LIFO is crucial for applications like undo/redo in text editors, managing nested function calls, and evaluating mathematical expressions in compilers.

Description of the stack operations

Operation name	Description
Push	Adds a new element (text) to the top of the stack
Pop	Removes and returns the top element of the stack
stackTop	Returns the top element without removing it
Display	Shows all elements in the stack
IsEmpty	Checks if the stack is empty
isFull	Only needed for array implementation. It checks if the stack is full

Chosen real-world application

Group 4 has chosen **"Undo/Redo functionality in text editors"** as its real-world application primarily due to its simplicity to show how stack functions. Undo and redo operations are fundamentally stack-based, which rely on the **Last In, First Out (LIFO)** principle. When an action is performed, it is pushed onto the Undo Stack. If "undo" is triggered, the top action is popped from the Undo Stack, the undo operation is executed, and the action is then pushed onto the Redo Stack. Similarly, when "redo" is triggered, the process is reversed, the top action is popped from the Redo Stack, the redo operation is executed, and the action is pushed back onto the Undo Stack. This seamless transfer between stacks efficiently manages the sequence of actions.

Comparative analysis of the implementations

In terms of ***memory usage***, the array-based method uses less memory compared to the pointer-based (linked list) method. An array element stores only the value, while a linked list node must store both the value and a pointer to the next node. However, arrays allocate a fixed amount of memory upfront, which may lead to wasted space if the stack is not fully utilized.

In terms of ***implementation***, Arrays are easier to implement than linked lists because they involve simple declarations and direct indexing. Each element has a unique index, which simplifies basic stack operations like push and pop. In contrast, the pointer-based stack requires dynamic memory management, including creating and deleting nodes, which adds complexity but provides greater flexibility.

In terms of ***scalability***, Scalability depends on the application's requirements. Arrays provide quick access to elements due to direct indexing, making them ideal for systems requiring fast performance with a predictable stack size. However, if the application involves frequent additions and removals or benefits from efficient memory usage, the pointer-based approach is better. The rule of thumb is to use array-based stacks for small-scale scenarios and pointer-based stacks for large-scale, dynamic applications.

Screenshots

- Array based approach screenshots

```
=== Text Stack Menu (ARRAY METHOD) ===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of Undo stack
0. Exit
Enter your choice (0-4):
```

Figure 1: Main menu of array-based approach

```
=== Text Stack Menu (ARRAY METHOD) ===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of Undo stack
0. Exit
Enter your choice (0-4): 1
Enter text: LUQMAN

Text added: "LUQMAN"
Current stack (most recent at the top):
LUQMAN

=== Text Stack Menu (ARRAY METHOD) ===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of Undo stack
0. Exit
Enter your choice (0-4): 1
Enter text: HARITH

Text added: "HARITH"
Current stack (most recent at the top):
HARITH
LUQMAN
```

Figure 2: Adding 2 elements to the stack with option 1, the first one is “LUQMAN”, the second is “HARITH”

```
Enter your choice (0-4): 4

Top of the stack: "HARITH"
```

Figure 3: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 2

Undid text: "HARITH"
Current stack (most recent at the top):
LUQMAN
```

Figure 4: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 2

Undid text: "LUQMAN"
Stack is empty.
```

Figure 5: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 2
Stack Underflow! No text to undo/redo.
Stack is empty.
```

Figure 6: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 3

Redid text: "LUQMAN"
Current stack (most recent at the top):
LUQMAN
```

Figure 7: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 3

Redid text: "HARITH"
Current stack (most recent at the top):
HARITH
LUQMAN
```

Figure 8: Option 4 will display the top element of the stack

```
Enter your choice (0-4): 3
Stack Underflow! No text to undo/redo.
Current stack (most recent at the top):
HARITH
LUQMAN
```

Figure 9: Option 4 will display the top element of the stack

- Pointer/Link list based approach screenshots

```
=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4):
```

Figure 10: Menu for Pointer-based approach

```
=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 1
Enter text: Hanis

Text added: "Hanis"
Current stack (most recent at the top):
Hanis

=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 1
Enter text: Syahirah

Text added: "Syahirah"
Current stack (most recent at the top):
Syahirah
Hanis
```

Figure 11: Inserting text with option number 1

```
Enter your choice (0-4):
4

Top of the stack: "Syahirah"
```

Figure 12: Displaying the top of the stack

```
=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 2

Undid text: "Syahirah"
Current stack (most recent at the top):
Hanis

=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 2

Undid text: "Hanis"
Stack is empty.
```

Figure 13: Undo-ing actions

```
Enter your choice (0-4): 2
Stack Underflow! No text to undo/redo.
Stack is empty.
```

Figure 14: Stack underflow occurs

```
=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 3

Redid text: "Hanis"
Current stack (most recent at the top):
Hanis

=== Text Stack Menu (LINKED LIST METHOD)===
1. Add text
2. Undo last text
3. Redo last undone text
4. View top of stack
0. Exit
Enter your choice (0-4): 3

Redid text: "Syahirah"
Current stack (most recent at the top):
Syahirah
Hanis
```

Figure 15: Redo-ing actions

```
Enter your choice (0-4): 3
Stack Underflow! No text to undo/redo.
Current stack (most recent at the top):
Syahirah
Hanis
```

Figure 16: Stack underflow